

SAP ABAP RAP (RESTful Application Programming Model) 後端開發課程

本課程教 **RAP** 的後端 **OData Service** 開發——資料模型 (CDS)、商業邏輯 (Behavior Definition : CRUD / Determinations / Validations / Actions)、服務發布 (Service Definition / Service Binding)。真正的 **Fiori Elements** 畫面設計 (**List Report / Object Page** 版面配置、**Draft** 編輯體驗) 不在本課程範圍內，但因為要銜接後續規劃中的 Fiori Elements 課程，**CDS View 的 @UI.* Annotation 語法 (Metadata Extension / `annotate view ... with { }`) 會在 rap02 教到基礎語法，讓學生知道怎麼寫、寫在哪裡，深入的畫面設計技巧留給 **Fiori Elements** 課程。課程全部完成 (rap01 ~ rap09，2026-08-17) **，rap08 起主要物件改由使用者在 Eclipse 建立/延伸、Claude 事後驗證；rap09 (期末 Capstone) 已驗收，整個課程正式結案。

⚠️ 開課前已實測查證的環境限制 (見 [.claude/rules/sap-adt-mcp.md](#) 第 40 節)：這個系統的 RAP 框架屬於較舊世代 (**Classic RAP**)，CDS 編譯器不支援 `define view entity` (新式 View Entity 語法)，BDEF 不支援 `strict` 子句——這兩者是目前 SAP 官方教材 / 認證預設使用的語法，本系統用不了。此外，**Service Binding** 這個物件一律要由使用者在 Eclipse 用官方精靈手動建立 + **Publish** (第 40.9 節已確認：用 ADT REST API 手動建的 Service Binding 缺少精靈才會觸發的後端註冊步驟，Publish 永遠失敗)，Claude 只能建到 Service Definition 這一層為止。✅ 已實測成功：Eclipse 精靈建立 + Publish + Preview 可以正常跑出 Fiori Elements List Report；發布成功後，**OData** 端點外網可以直接用瀏覽器/Postman 測試 (只要用系統對外的正確主機名稱 + Port，不需要內網或 VPN——這點連帶更正了 REST 課程原本記載的「只能內網測試」，見第 15 節 2026-08-02 更正)。這些差異會在 rap01 開頭完整說明，之後每一課遇到都會再次提醒。

⚠️ ⚠️ rap03 發現的重大限制 (見 [.claude/rules/sap-adt-mcp.md](#) 第 43 / 44 節)：這個系統的 **RAP Managed Runtime** (CUD 寫入部分) 被 SAP 官方標記為「尚未對外釋出」，任何 **Managed BDEF** 的 `create/update/delete` 執行到底層一律 **Dump** (`CL_CSP_MD_METADATA_FACTORY` 的套件白名單機制，程式碼裡留著開發者自己寫的註解「csp isn't released for public usage until now」；SAP Community 一篇 2019-11 的貼文一字不差回報同樣問題，時間點跟這套系統的 S/4HANA 1909 高度吻合)。因此課程從 rap03 起改成 **Managed / Unmanaged** 兩者都教：Managed 語法當知識儲備 (銜接未來 ABAP Cloud RAP 課程)，**Unmanaged** 是這系統上唯一能真正端對端執行、驗證的版本 (已用 EML + `programrun` 完整驗證成功)。這個發現也連帶更正了先前「EML 沒辦法無頭驗證」的誤判——問題從來不是 EML，是 Managed Runtime 的致命 Dump 透過 RFC Bridge 傳回時卡住了連線。

課程定位

- 對象：完成基礎課 ([src/ABAP_Training/](#)) ex08 模組化、ex15 CHANGING+Table Type 的學員，需要能看懂 DDIC Table / CDS View / Class 的基本語法；不要求先修 OOP 課程，但有 OOP 基礎 ([src/ABAP_Training_OOP/](#)) 會更容易理解 Managed BDEF 背後「框架自動生成 Runtime 邏輯」的概念。
- 技術範圍：DDIC Table → CDS Interface View (含基礎 @UI.* Annotation 語法，為銜接 Fiori Elements 課程鋪路) → Behavior Definition
Managed 與 Unmanaged 兩者對照教學 (CRUD / Determinations / Validations / Actions / Associations，Managed 當語法知識儲備、Unmanaged 是這系統上真正能執行驗證的版本，見下方「Managed 與 Unmanaged 並行教學」) → Service Definition → Service Binding (OData V2 為主，V4 只教概念不強求發布)。不含：Fiori Elements 完整畫面設計 (List Report / Object Page 版面配置技巧、Draft 編輯體驗——這些留給後續規劃中的 Fiori Elements 課程) / 真正的 ABAP Cloud 語言版本 RAP (見下方「與未來課程的關係」)。
- 結業標準 (草案)：能獨立設計一個 Header-Item 兩層式 Managed RAP BO (含 Determination 自動衍生欄位、Validation 擋非法資料、一個自訂 Action)，寫出正確的 CDS / BDEF / Service Definition，並在系統允許的範圍內驗證服務可以發布。

與未來課程的關係：這是「Classic RAP」，不是「ABAP Cloud RAP」

RAP 框架本身從 SAP 導入以來就分成兩種寫法：

- Classic RAP** (本課程教的)：CDS 用舊式 `define view` (無 `entity` 關鍵字)，BDEF 無 `strict` 模式，可以存取任何 DDIC 物件與 ABAP 語法，不受 ABAP Cloud 語言限制——這是目前這個系統唯一能寫得出來、啟用得了的版本 (已實測驗證，見下方 rap01)。
- ABAP Cloud RAP** (SAP 現在主推、官方教材/認證的預設版本)：CDS 用新式 `define view entity`，BDEF 支援 `strict(2)` 等新語言特性，且必須搭配 ABAP Cloud 限制語法 (只能用 Released API，見下方 rap01 對這個限制的完整說明)，套件要啟用 ABAP Cloud 語言版本——這需要 S/4HANA 2022 以後的 On-Premise 系統，或 SAP BTP ABAP Environment (Cloud)。

這個落差不是本課程能解決的 (受限於目前連線的系統版本)，待日後設定好一個支援 **ABAP Cloud** 語言版本的系統 (**Cloud ABAP Environment** 或更新版 **On-Premise**)、建立對應的 MCP 連線之後，會另開一門課專門教 **ABAP Cloud RAP**，把兩者的語法差異、遷移注意事項講清楚。本課程結束時學生會清楚知道「我學的是哪個版本、跟官方教材的落差在哪裡」，不會誤以為兩者完全一樣。

Managed 與 Unmanaged 並行教學 (rap03 起的重大調整)

原本規劃只教 Managed (見上面「不含 Unmanaged RAP」的舊決定)，但 rap03 出題時發現：這系統的 **RAP Managed Runtime** (CUD 寫入部分) 被 SAP 官方標記為「尚未對外釋出」——`CL_CSP_MD_METADATA_FACTORY` 這個類別會檢查 CDS Entity 所在的套件是否在一份硬編碼白名單裡 (`SB0I_RAP_CSP_TST%` 等 SAP 內部套件)，不在清單裡就用致命訊息 (`MESSAGE ... TYPE 'X'`) 擋下來，程式碼裡甚至留著開發者自己的英文註解「csp isn't released for public usage until now」。SAP Community 一篇 2019-11 的貼文一字不差回報同樣問題，時間點跟這套系統的 **S/4HANA 1909** (Eclipse Project 顯示的系統代號 `S4D_1909`) 高度吻合，判斷是這個版本世代 Managed Runtime 寫入功能尚未對客戶套件開放的已知限制 (詳細技術細節見 [.claude/rules/sap-adt-mcp.md](#) 第 43 節)。

因應調整：從 rap03 起，**Managed 與 Unmanaged** 兩者都教，並列比較：

- Managed**：語法教學 + 能啟用，但明確告知「這系統執行不了」，當作知識儲備——語法跟官方教材 / 未來的 ABAP Cloud RAP 課程一致，銜接用

- **Unmanaged**：語法教學 + 已用 **EML + programrun 完整驗證成功**（含資料庫寫入確認），這系統上真正能讓學生看到「資料真的寫進去了」的路徑；份量比原計畫重（需要自己寫實作類別的 **LOCK/CREATE/READ** 方法），但既然 **Managed** 走不通，這是必要的取捨

這個發現還有一個意外收穫：原本第 42 節記錄的「EML 沒辦法用 **programrun** 無頭驗證」是誤判——用 **Unmanaged BDEF** 測試完全無頭驗證成功，證實問題從來不是 EML 這個語言機制，是 **Managed Runtime** 的致命 Dump 透過 **RFC Bridge** 傳回時卡住了連線（已更正）。

對 **rap05 ~ rap07** 的影響：Determination / Validation / Action 在 **Unmanaged** 模式下沒有 **Managed** 那種宣告式語法（**determination ... on save** 這類），邏輯要自己寫在實作類別裡——這幾課出題時要重新評估教法，屆時再定案。

教學分工原則（2026-08-02 定案）：物件建立責任從 rap04 起逐步移轉給使用者

前期查證與 rap01 ~ rap03 出題階段，Table / CDS View / BDEF 這類物件都是 Claude 用 ADT API（原生工具或本檔記錄的 **curl workaround**）自動建立、寫入、啟用，使用者只需要在 Eclipse 對著已建好的物件做確認。但這樣的教法有個缺口：這系列課程的目標是讓學生學會在 **Eclipse ADT** 裡實際開發 **RAP**，不是只看懂 **Claude** 產生的程式碼——如果自始至終都由 Claude 全自動建立，學生不會練到「用 Eclipse New Wizard 建 RAP 物件」這個真實工作中最常用的操作手感（實務上多數 RAP 開發者是用 Eclipse 精靈產生骨架、再手動編輯內容，不是直接寫 REST XML）。因此跟使用者討論後定案分工如下：

- **Service Binding 建立 + Publish**：一律使用者在 **Eclipse ADT** 手動操作（技術上本來就必要，見上方「已實測查證的環境限制」與第 40.9 節：ADT REST API 手動建的 Service Binding 缺少精靈才會觸發的後端註冊步驟，Publish 永遠失敗）。
- **Table / CDS View / BDEF**：**rap01 ~ rap03** 由 **Claude** 建立示範（已完成）；自 **rap04** 起，題目若需要新建這類物件，改為使用者在 **Eclipse ADT** 手動建立，Claude 負責事後驗證（**sap_get_source** / GET 讀回內容比對）、語法檢查（**checkruns**）與除錯，不再自動用 API workaround 建立。
- **講義撰寫要求**：凡是改由使用者手動操作的步驟，題目 md 裡要寫清楚 **Eclipse ADT** 裡的詳細操作路徑（哪個選單、精靈畫面有哪些欄位、要填什麼值、按鈕順序），不能只寫「請在 Eclipse 建立 XXX 物件」這種籠統描述——目標是讓學生對照講義就能操作，不需要另外查文件。
- **校正機制**：若講義描述的操作步驟跟使用者實際看到的 Eclipse 畫面有落差（版面配置、欄位名稱、精靈流程跟記載的不一樣），由使用者回報差異，Claude 再修正講義內容——這個「使用者截圖/描述回報→修正講義」模式在 **Smartform** 課程已經驗證有效（見 **.claude/rules/sap-adt-mcp.md** 第 19 節與 [[feedback-verify-tcodes]] 記憶），沿用同一套做法。

教材慣例（比照 OOP/REST/AMDP/Enhancement 課程）

- 每題三件套：題目 **rapNN_主題.md** + PDF 講義（**node tools/md2pdf.js src/ABAP_Training_RAP**）+ 答案快照
- 每題 md 開頭（**## 學習目標** 之前）要有 **## Lecture** 完整背景知識講解
- 物件命名慣例（沿用查證階段已驗證可行的模式）：
 - DDIC Table：**ZRAPnn_<實體>**（如 **ZRAP02_TASK**）
 - 欄位型別（見 **.claude/rules/abap-style.md** 硬性規則）：語意對應標準表既有欄位一律直接引用標準 Data Element（如 **TEXT100**），先用 **quickSearch** 查證不要憑記憶猜；沒有合適標準 DE 才自己建 **ZRAPnn_<欄位>** 命名的 Domain + Data Element（同名），不可以留著 **abap.char(...)** 這類內建型別
 - CDS Interface View：**ZI_RAPnn_<實體>**，Behavior Definition 直接綁在同名 CDS View 上（本系統是單層模式，Interface View 直接掛 BDEF，不強制要有獨立的 Projection View 兩層架構，除非題目要示範多消費場景）
 - Metadata Extension（UI Annotation）：跟系統既有標準物件命名慣例一致，**直接沿用同名**（如 **ZI_RAPnn_<實體>**）——DDLX 跟 DDLS 是不同物件型別，同名不衝突，Eclipse 建立精靈選擇要擴充的 CDS View 之後會自動帶出這個名稱
 - Service Definition：**ZRAPnn_SD**；Service Binding：**ZRAPnn_SB**
 - EML / 其他驗證程式：**ZR_RAPnn_DEMO**（Managed 版本）/ **ZR_RAPnn_DEMO_UM** 或類似後綴（Unmanaged 版本，沿用 sf / rs 課程 **ZR_<課程代碼>nn_DEMO** 的命名慣例再加註記）
 - **Unmanaged 實作類別**：**ZBP_I_RAPnn_<實體>**，Global 類別本體只是空殼 + **FOR BEHAVIOR OF <view>** 子句，真正的 Handler 邏輯寫在 Local Types Include（PUT 到 **/sap/bc/adt/oo/classes/<class>/includes/implementations**，可以完全交給 ADT API 自動建立，不需要 Eclipse 精靈，見第 44 節）
- 每題原始碼快照：Table 用 **.tab1.abap**、CDS View 用 **.ddls.abap**（沿用第 16 節 DDLS 快照慣例）、Metadata Extension 用 **.ddlx.abap**、BDEF 用 **.bdef.abap**、Service Definition 用 **.srvd.abap**、驗證程式用 **.prog.abap**、Unmanaged 實作類別用 **.clas.abap**（Global 本體）+ **.clas.locals_imp.abap**（Local Handler 實作，沿用 abapGit 慣例）；Domain / Data Element 是結構化物件（非 source-based），存 **.doma.xml** / **.dtel.xml**；Service Binding 同樣結構化（見查證階段第 40.5 節），存 **.srvb.xml**
- **✓ 已更正（第 42 / 44 節）**：**EML 完全可以用 programrun 無頭驗證**——原本第 42 節記錄「EML 沒辦法無頭驗證」是誤判，真正卡住的原因是 **Managed Runtime** 的致命 Dump（見上方「Managed 與 Unmanaged 並行教學」），**Unmanaged BDEF** 的 EML 已用 **programrun** 完整驗證成功。**Managed BDEF** 的 EML 驗證程式則反過來——不要嘗試用 **programrun** 或請使用者在 **SAP GUI** 執行，一律會 **Dump**，這是預期中的已知限制，只需確認語法正確、成功啟用即可

課綱（規劃中，rap01 ~ rap08 已出題）

#	主題	內容重點	銜接前面課程	狀態
rap01	為什麼用 RAP？環境限制聲明	RAP 在 SAP 開發架構中的定位（對比 Function Module / BAPI / classic report / 既有課程學過的技術）；Managed vs Unmanaged 概念對照（系統既有標準物件	呼應基礎課 ex08 模組化、REST 課程（ src/ABAP_Training_REST/ ）的服務導向概念	已出題

#	主題	內容重點	銜接前面課程	狀態
		<code>C_SalesOrderManage</code> 就是 Unmanaged ; ⚠️ 原本規劃只教 Managed · rap03 發現這系統 Managed CUD 執行不了後改成兩者並教 · 見 README 「Managed 與 Unmanaged 並行教學」) ; 完整說明已查證的環境限制 : Classic RAP vs ABAP Cloud RAP 語法差異 (無 <code>view entity</code> 、無 <code>strict</code>) 、ABAP Cloud 限制語法是什麼 (Released API / 禁用 Classical Dynpro 等) 、OData V2 vs V4 發布能力差異 ; RAP 五層架構總覽 (Table→Interface View→BDEF→Service Definition→Service Binding)		
rap02	CDS Interface View 資料模型基礎 + Metadata Extension (UI Annotation) 入門	Part A - 資料模型 : Eclipse ADT 建立 Transparent Table Step by Step ; DDIC Table 設計 (沿用第 34/39 節的 annotation 慣例) ; DDL 語法深入 (<code>key</code> / <code>not null</code> / Foreign Key cardinality / Currency-Quantity 欄位 <code>@Semantics.amount.currencyCode</code> / <code>@Semantics.quantity.unitOfMeasure</code>) ; Primary / Secondary Index 概念 + Eclipse ADT 建立步驟 + SQL Console EXPLAIN PLAN 驗證 (練習 1 : 使用者自己建一個 Secondary Index) ; 欄位型別一律引用 Data Element 的硬性規則——有標準 DE 直接重用 (<code>description</code> 用 <code>TEXT100</code>) · 沒有合適標準 DE 就自建 Domain + DE (<code>task_id</code> / <code>status</code> / <code>priority</code> / <code>due_date</code> 各建一組 · <code>status/priority</code> 示範固定值清單語法 · 並澄清固定值只在 UI 層生效不是資料庫約束) ; CDS 舊式 <code>define root view</code> (本系統適用版本) ; 必要 annotation : <code>@AbapCatalog.preserveKey</code> / <code>@ObjectModel.compositionRoot</code> / <code>@AccessControl.authorizationCheck</code> ; CDS View 的 Client Handling 自動機制 (底層 <code>mandt</code> 型別欄位不用列進 View · 靠 <code>\$session.client</code> 自動過濾 · 對照 AMDP 完全不自動處理 Client) ; 欄位別名與 <code>mapping for</code> 的取捨 ; Eclipse ADT 建立 CDS View Step by Step ; CDS View 分類 (<code>I_</code> Interface / <code>C_</code> 或 <code>P_</code> Consumption-Projection / <code>R_</code> · RAP 用 Interface View) ; Association vs. Join (Association 只有被 path expression 引用才轉成 Join · 是 RAP Composition 的基礎) ; Built-in Functions (字串 / 數值 / 日期時間 / 條件) ; Parameters & Session Variables (<code>with parameters</code> / <code>\$parameters.pname</code> / <code>:pname</code> / 內建 <code>\$session.*</code> 清單 · 官方文件查證) (練習 2 : 使用者自己建一個帶 Association 的 CDS View · 用標準表 SPFLI/SCARR) 。**Part B - Metadata Extension (獨立小節 · 第 40.10 節已查證 : <code>@UI.*</code> 跟 Classic/ABAP Cloud RAP 語法版本無關 · 這個系統完全支援) : CDS View 要先開 <code>@Metadata.allowExtensions: true</code> 才能被擴充 ; Metadata Extension (DDLX) 是獨立物件** · 跟 CDS View 同名但不同物件型別 (<code>DDLX/EX</code>) ; 本系統適用的舊式語句 <code>annotate view ZI_XXX with { ... }</code> (不是新式的 <code>annotate entity</code>) ; 教三個最常	承基礎課 DDIC 表格設計、AMDP 課程 CDS Table Function 的 DDLS 建立流程	已出題 (<code>ZRAP02_TASK</code> 表格 + 4 組 Domain/DE + <code>ZI_RAP02_TASK</code> View + Metadata Extension 均已建立啟用 · <code>sap_inactive_objects</code> 確認 0 筆殘留 ; 2026-08-02 應使用者要求大幅擴充 Table / Index / CDS View 進階語法教學 + 兩個動手練習 · 內容已用 sap-docs 官方文件查證)

#	主題	內容重點	銜接前面課程	狀態
		用的 <code>@UI.*</code> 標記——<code>@UI.headerInfo</code> (Object Page 標題) / <code>@UI.lineItem</code> (List Report 表格欄位) / <code>@UI.selectionField</code> (篩選欄位) ; 這一節只教語法本身跟寫在哪裡 (DDLX 物件怎麼建、<code>annotate</code> 語句怎麼寫) , 不深入 List Report / Object Page 版面設計邏輯 , 完整畫面配置留給 Fiori Elements 課程		
rap03	Behavior Definition 基礎——Managed 與 Unmanaged 對照	Part A - Managed (知識儲備) : <code>managed</code>; <code>header</code> (本系統不支援 <code>strict</code>) ; <code>persistent table</code> / <code>lock master</code> ; CRUD 操作宣告 ; <code>field (readonly)</code> / <code>field (mandatory)</code> 欄位控制 ; ETag 語法 (第 40.11 節 : 這系統是 <code>etag <欄位></code> , 不帶 <code>master</code>) ; 語法正確但這系統執行不了 (第 43 節 : <code>CL_CSP_MD_METADATA_FACTORY</code> 套件白名單機制 , Managed Runtime 尚未對客戶套件開放 , 判斷跟 S/4HANA 1909 版本有關 , SAP Community 2019-11 貼文佐證) 。 Part B - Unmanaged (這系統真正能跑的版本) : <code>implementation unmanaged in class ... unique</code>; (無 <code>persistent table</code>) ; 實作類別繼承 <code>cl_abap_behavior_handler</code> , <code>FOR LOCK/FOR MODIFY/FOR READ</code> 方法綁定語法 (照抄標準物件 <code>CL_SD_BEHV_SALESORDERMANAGE</code> 的真實寫法) ; 已用 EML + programrun 完整驗證成功 (資料真的寫進資料庫 , 且證實 EML 本身完全支援無頭執行 , 第 42 節原本的「EML 會卡住」誤判已更正 , 真正原因是 Managed Runtime Dump) 。 Part C : Managed vs Unmanaged 差異總表	承 rap02	已出題 (Managed : <code>ZI_RAP02_TASK BDEF + ZR_RAP03_DEMO</code> ; Unmanaged : <code>ZRAP03_UMTEST / ZI_RAP03_UMTEST / ZBP_I_RAP03_UM4 / ZR_RAP03_UMTEST</code> , 全部建立啟用 , Unmanaged 已驗證執行成功)
rap04	Service Definition / Binding 與發布流程	<code>define service / expose ... as ...</code> ; Service Binding 一律由使用者在 Eclipse 用官方精靈手動建立 (第 40.9 節已確認 : 用 ADT REST API 手動建的 Service Binding 缺少精靈才會觸發的後端註冊步驟 , Publish 永遠失敗且錯誤訊息極具誤導性 , Claude 不再嘗試 API workaround) ; Service Definition 由 Claude 建立與驗證 (沿用既有 rap02/rap03 的 CDS View , 本課不需要新建 Table / CDS View / BDEF , 故【教學分工原則】的移轉暫不影響本課 , 從下一課開始若需要新建這類物件才適用) ; Service Binding 建立 + Publish 是操作指引 , 講義要寫清楚 Eclipse 精靈每一步的畫面與欄位 , 由使用者對照操作、截圖回報 (比照 Smartform 課程模式) ; OData V2 技術服務名稱 = Binding 物件名稱 ; 已實測成功 : Eclipse 精靈建立 + Publish + 內建 Preview 開出 Fiori Elements List Report ; 就算發布成功 , 外部 (Postman/瀏覽器) 連線是否可達仍取決於使用者當下網路位置 , 需要每次確認 ; 同一個 Service Definition 刻意混搭 rap02 的 Managed CDS View 與 rap03 的 Unmanaged CDS View (別名 <code>TaskManaged</code> / <code>TestUnmanaged</code>) , 讓學生在同一個 Fiori Elements 預覽畫面對照「Managed 唯讀、Unmanaged 讀寫都正常」的差異	承 REST 課程對 OData / 服務發布概念的鋪墊、rs07 的 <code>cl_http_client</code> 自我呼叫驗證手法 (<code>destination = 'NONE'</code> , 已用 <code>ZR_RAP04_SELFTEST</code> 實作)	✅✅ 已出題並完整驗收結果 (Service Definition / Service Binding / Publish 全部確認成功 ; <code>TaskManaged</code> (Managed) Fiori Elements Preview 讀取正常、Create 觸發預期中的 Dump ; <code>TestUnmanaged</code> (Unmanaged) 端對端 Create 真的成功 , 新資料確實寫進資料庫——過程中補齊了 rap03 遺漏的 <code>ZI_RAP03_UMTEST</code> Metadata Extension / 欄位標籤才讓畫面正常顯示 ; <code>ZR_RAP04_SELFTEST</code> 自我呼叫驗證程式的讀取部分已用 SE38 驗證成功 , 寫入部分因 CSRF 自我呼叫限制放棄、改以 Eclipse Preview 為準 , 第 45 節已完整記錄排查過程)

#	主題	內容重點	銜接前面課程	狀態
rap05	Determinations (自動衍生欄位)	Part A (Managed · 語法知識儲備) : <code>determination <名稱> on save { create; }</code>; BDEF header 一旦有 Determination 就要改成 <code>managed implementation in class ...</code> <code>unique;</code>; ⚠ 這系統的 Handler Method 要用 <code>obsolete</code> 語法 <code>FOR DETERMINATION <alias>~<det名稱></code> (不是官方新式 <code>FOR DETERMINE ON SAVE</code>) · <code>IMPORTING keys FOR <alias></code> (不重複 <code>~det名稱</code>) · 逐步排錯過程 (6 個錯誤訊息) 完整記錄在講義; <code>READ ENTITIES/MODIFY ENTITIES ... IN LOCAL MODE</code> 是 Determination Handler 專屬 EML; ⚠ 這系統這個語法組合下 <code>field(readonly)</code> 會擋住 Determination 內部寫入 (跟官方範例行為不同) · <code>TIMESTAMPL</code> 需要 <code>cl_abap_tstmp=>utclong2tstmp()</code> 明確轉換。Part B (Unmanaged · 這系統真正能跑) : 查證官方文件 <code>ABENBDL_DETERMINATIONS</code> 確認「Not available for unmanaged, non-draft RAP BOs」是官方明講的限制 (不是這系統的問題) ——沒有宣告式語法 · 改用「私有方法 + 在 <code>CREATE</code> 裡明確呼叫」的手寫模式 · <code>ZRAP03_UMTEST</code> 延伸加上 <code>created_at/created_by</code> 欄位 · 已用 <code>programrun</code> 完整驗證成功 (自動填值正確)。Part C : Managed vs Unmanaged 差異總表	承 rap03	已出題並驗證 (Managed : 延伸 <code>ZI_RAP02_TASK</code> BDEF + 新增 <code>ZBP_I_RAP02_TASK</code> 實作類別 + <code>ZR_RAP05_DEMO</code> ; Unmanaged : 延伸 <code>ZRAP03_UMTEST / ZI_RAP03_UMTEST</code> (Table/CDS View/Metadata Extension/BDEF) + <code>ZBP_I_RAP03_UM4</code> + <code>ZR_RAP05_DETDEMO</code> · 全部建立啟用 · Unmanaged 已用 <code>programrun</code> 驗證執行成功)
rap06	Validations (資料完整性檢查)	Part A (Managed · 語法知識儲備) : <code>validation <名稱> on save { field <欄位>; }</code> (只能 <code>on save</code> · 沒有 <code>on modify</code>) ; Handler Method 同樣要用 <code>obsolete FOR VALIDATION <alias>~<val名稱></code>; 官方文件明講失敗時「整個 RAP 交易全部拒絕」(跟 Determination 不同 · Determination 本身不會讓交易失敗) ; 新發現 : 單一實體 BDEF 下 · <code>FOR DETERMINATION/FOR VALIDATION</code> 的 <code>failed/reported</code> 直接是表格 · 但 <code>FOR MODIFY (create)</code> 的要用 <code>-<alias></code> 分類 · 兩者不一致要小心。Part B (Unmanaged · 這系統真正能跑) : 查證官方文件確認 Validation 跟 Determination 適用同一條限制 (非 Draft Unmanaged 不支援) ; 等效寫法在 <code>CREATE</code> 裡檢查後 <code>APPEND</code> 到 <code>failed-test/reported-test</code> ; 已用 <code>programrun</code> 驗證成功 · 且意外揭露 Managed / Unmanaged 在「部分失敗」語意上的行為差異 (Managed 整批拒絕 · 這一課的 Unmanaged 逐列處理範例是合法資料照樣寫入)。Part C : 差異總表	承 rap05	已出題並驗證 (Managed : 延伸 <code>ZI_RAP02_TASK</code> BDEF + <code>ZBP_I_RAP02_TASK</code> 加 <code>validateStatus</code> ; Unmanaged : 延伸 <code>ZBP_I_RAP03_UM4 create</code> 方法加驗證邏輯 + 新建 <code>ZR_RAP06_VALDEMO</code> · 已用 <code>programrun</code> 驗證成功)
rap07	Actions (自訂操作)	Part A (Managed · 語法知識儲備) : <code>action <名稱> result [1] \$self;</code> ; Handler Method <code>FOR MODIFY ... FOR ACTION <alias>~<action名稱> RESULT result</code>——⚠ 重大發現 : Action 不需要 <code>obsolete</code> 語法 · 直接套用官方現行語法一次	承 rap03/rap05/rap06	已出題並驗證 (Managed : 延伸 <code>ZI_RAP02_TASK/ZBP_I_RAP02_TASK</code> 加 <code>markDone</code> ; Unmanaged : 延伸 <code>ZI_RAP03_UMTEST/ZBP_I_RAP03_UM4</code> 加 <code>touch</code> + 新建 <code>ZR_RAP07_ACTDEMO</code> · 已用

#	主題	內容重點	銜接前面課程	狀態
		就編譯成功（因為 Action 掛在 FOR MODIFY 底下跟 CREATE/UPDATE/DELETE 共用類別，不像 Determination/Validation 各自有專屬 Handler 類別）。 Part B (Unmanaged) ，這系列課程第一次有真正宣告式語法可用：查證官方文件確認 Action 沒有「非 Draft 不支援」限制（跟 Determination/Validation 不同），Managed / Unmanaged 都直接支援；已用 programrun 完整驗證：EML EXECUTE <action> FROM VALUE #(...) 呼叫、資料真的被改、 RESULT （透過 %param ）正確回傳，不是手寫等效版本。 Part C ：差異總表 + rap05 ~ rap07 三課「obsolete 語法 / Draft 限制」全貌回顧		programrun 驗證 created_at 真的被重新蓋章)
rap08	Associations / Compositions (多層結構)	⚠️ 教學分工原則正式落地：這是第一課主要物件（兩張表 / 兩個 CDS View / BDEF / 實作類別）改由使用者在 Eclipse 手動建立，Claude 負責事後驗證與除錯，講義附完整 Eclipse Step by Step。Claude 先用暫時性驗證物件把語法在系統上實測過一輪： composition [0..*] of / association to parent (Managed / Unmanaged 語法相同，只有實作邏輯不同)；子實體 lock 要用 lock dependent (欄位 = 欄位) (不是官方教材的 lock dependent by _別名)；Create-by-Association Handler 用 FOR CREATE <alias>_<association>；重大發現：父實體 Key 是使用者自訂值時，EML 呼叫端要用 %key-<欄位> 連結子實體建立，用官方範例常見的 %cid_ref 會靜默失敗（無錯誤訊息但子實體建不出來），已用 programrun 驗證 %key 版本端對端成功（Header + 2 筆 Item 一次建立）。使用者實際在 Eclipse 建立六個物件後，Claude 發現實作類別骨架漏了對應 Item 的 lhc_item Handler 類別（原講義只補了 lhc_header），已補上 read_item（完整實作）+ update / delete（留空，列入練習題）。Part D 加碼：補上 RAP 測試「Behavior / BO 層」（EML）vs.「Service / OData 層」（真正 OData 呼叫）兩層架構說明，並建立 Service Definition ZRAP08_SD（expose Composition Root，別名要避開 SQL 保留字如 ORDER）。Service Binding 一律使用者 Eclipse 建立 + Publish（技術上必須）	承 rap02 ~ rap07 全部技巧	✅ 已出題並驗證 (2026-08-17)：Behavior 層——ZR_RAP08_DEMO 用 programrun 驗證 Create-by-Association 一次呼叫建立 Header + 2 筆 Item 成功，READ ENTITIES 讀回資料正確；Service / OData 層（Part D）——ZRAP08_SD 已建立並啟用，Service Binding ZRAP08_SB 待使用者 Eclipse 建立 + Publish 後 Claude 驗證
rap09	期末綜合實作 (Capstone)	延伸 rap08 已建好的「訂單」Header-Item BO（不重新設計資料模型），補上 Determination-equivalent（沿用既有 create 自動填值，多填 status）、Validation-equivalent（quantity 必須 > 0，手寫邏輯，因為 BDEF 的 validation 宣告式語法在 Unmanaged 非 Draft 不支援，已實測確認）、Action（confirmOrder，帶「空訂單不能確認」商業規則，RESULT 要用 %key + %param 巢狀賦值）。加碼更新 Metadata Extension 讓 Preview 出現「Confirm Order」按鈕；Part D 再加碼：實測發現這系統（S/4HANA 1909 + Unmanaged 非 Draft）Object Page 子表格標準 Create 按鈕不可靠	呼應各課程 Capstone 慣例（sf06 / ex28），整合 rap05 ~ rap08 全部技巧	✅ 已出題並驗證 (2026-08-17)：使用者於 Eclipse 延伸四個 rap08 物件，ZR_RAP09_DEMO 用 programrun 驗證三個情境 + Part D 的 addItem 情境全部成功，Service Definition / Metadata Extension 都已更新並啟用

#	主題	內容重點	銜接前面課程	狀態
		(Edit 模式異常) · 改用帶參數的 Custom Action (<code>addItem</code> · 第一次教 <code>action ... parameter <Abstract Entity></code> 語法) 繞開 · 並回頭修正 rap08 講義「Composition 子實體不用 expose」的錯誤推論 (實測必須 expose 才會有 Navigation Property)		

開課前查證的完整技術細節 (含每個踩到的錯誤訊息、workaround 語法) 記錄在 `.claude/rules/sap-adt-mcp.md` 第 40 節 · rap01 會摘要說明給學生 · 出後續題目時遇到新狀況會持續補充該節。